

TASK-1 QnA

1. Where is the RISC-V program located in the vsd-riscv2 repository?

The programs are located in the `samples` folder of the repository. Key files present in this directory include:

- `sum1ton.c`
- `1ton_custom.c`
- `load.S`
- `Makefile`

2. How is the program compiled and loaded into memory?

The C source is compiled using `riscv64-unknown-elf-gcc` (SiFive prebuilt GCC 8.3.0, installed at `/opt/riscv/bin`) into an ELF binary. It is then loaded and executed via the Spike ISA simulator using the proxy kernel (`pk v1.0.0`):

```
spike pk sum1ton.o
```

For actual FPGA deployment (`vsdfpga_labs`), the firmware is compiled into a `.hex` file and loaded into block RAM:

```
make riscv_logo.bram.hex
```

The bitstream can be then flashed to the FPGA with `make flash` command.

3. How does the RISC-V core access memory and memory-mapped IO?

In simulation, Spike provides a virtual memory model; the proxy kernel (`pk`) handles syscalls (such as `printf`) by proxying them to the host.

On FPGA, the core accesses block RAM for instruction and data memory. Peripheral communication is handled via memory-mapped IO — UART output is communicated physically through the CH340 USB-UART bridge on pins 3 (TX) and 4 (RX), and is observed via the serial terminal (`make terminal` using `picocom` on `/dev/ttyUSB0`).

4. Where would a new FPGA IP block logically integrate in this system?

A new FPGA IP block would integrate in the RTL layer, specifically within `vsdfpga_labs/basicRISCV/RTL`, alongside the existing core and memory.

It would be connected to the RISC-V core's memory bus as a memory-mapped peripheral with a fixed base address. Firmware would be updated to access registers at that address. The standard build and deployment pipeline would then apply:

- `make clean`
- `make build`
- `make flash`